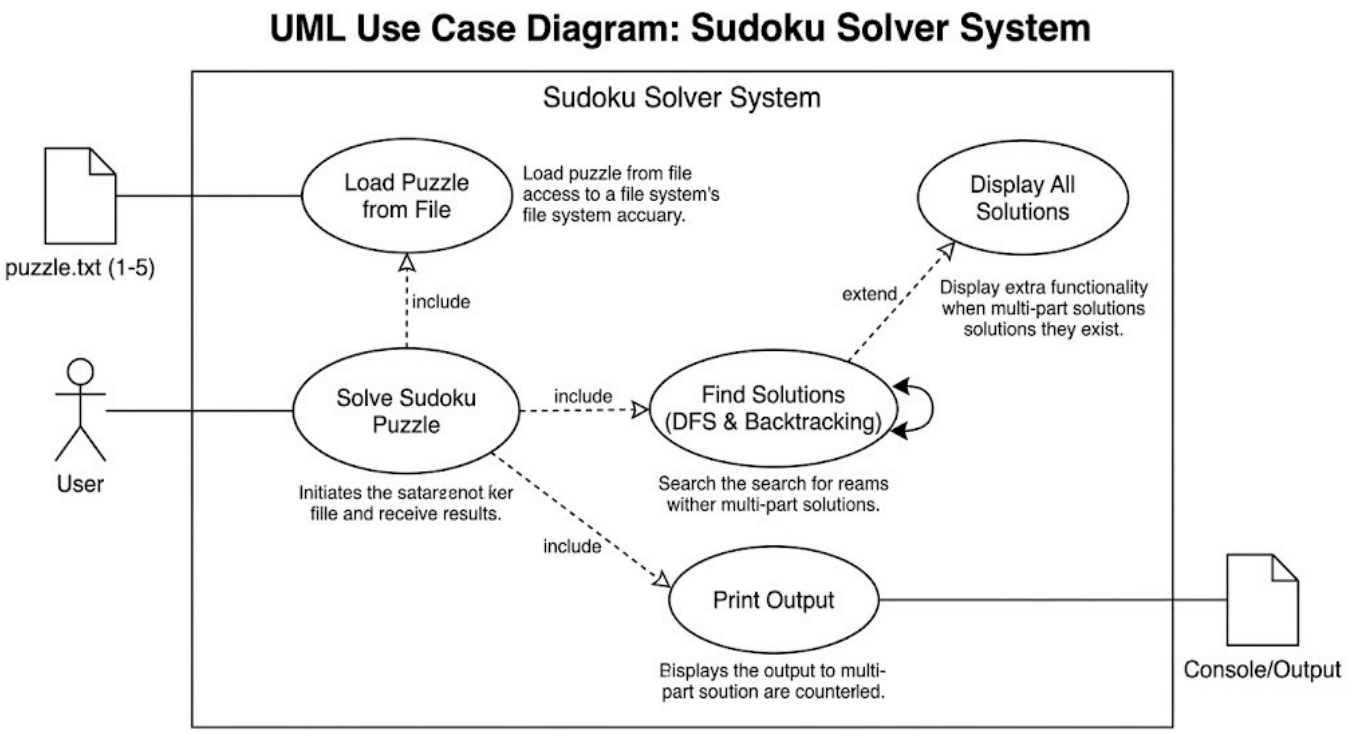


Use Case 1: Solve Sudoku Puzzle

- **Description:** The primary use case that serves as the entry point for the user. It orchestrates the entire process of solving a given Sudoku puzzle file. The diagram correctly shows it as including several other use cases: 'Load Puzzle from File', 'Find Solutions', and 'Print Output'.
- **Actor:** User
- **Step-by-Step Outline:**
    1. User starts the Sudoku solver program.
    2. User selects one of the five puzzle files (e.g., `puzzle1.txt`).
    3. System initiates the 'Load Puzzle from File' use case to read the puzzle data.
    4. System initiates the 'Find Solutions' use case to search for potential solutions.
    5. System initiates the 'Print Output' use case to display the results to the user.
    6. The system terminates or waits for the user's next action.
- **Detailed Description:** This is the main use case that the User directly interacts with. It begins when the user triggers the program. The system then prompts for or automatically selects a file name, as specified (puzzle1.txt through puzzle5.txt). This use case acts as a controller, directing the flow of execution by leveraging the specialized functionalities of the included use cases. It first calls 'Load Puzzle from File' to get the initial puzzle board, then invokes 'Find Solutions' to apply the backtracking algorithm, and finally, utilizes 'Print Output' to present the original puzzle, the solved version, and any multi-part solutions, or reports that no solution was found.

Use Case 2: Load Puzzle from File (Included by "Solve Sudoku Puzzle")

- **Description:** This use case handles the interaction with the external file system to read the content of the specified Sudoku puzzle file. It is a necessary precursor to the solution process.
- **Actor:** System (internal action, can be viewed as an internal system event in UML, though often shown connecting to a "File System" secondary actor in more complex systems).
- **Step-by-Step Outline:**
    1. The use case receives the file name (e.g., `puzzle1.txt`) from the including 'Solve Sudoku Puzzle' use case.
    2. System attempts to locate and open the specified file within the same directory as the program.
    3. System parses the file content, reading the 9x9 grid, interpreting digits as filled-in cells and underscores as empty cells.
    4. If successful, the use case returns the parsed puzzle representation (e.g., an object-oriented grid model) to the calling use case.
    5. If unsuccessful (e.g., file not found, permission denied), an error is reported back to the system, and the primary use case handles the failure.
- **Detailed Description:** This use case encapsulates all file system operations. It is critical for the program to correctly interpret the puzzle data, translating the underscore characters into the system's internal representation for blank squares. The diagram correctly separates this I/O-heavy task from the core algorithmic logic, improving modularity. The 'puzzle.txt (1-5)' outside the system boundary reinforces that this use case accesses external resources, which might fail, and must handle such potential errors.

Use Case 3: Find Solutions (DFS & Backtracking) (Included by "Solve Sudoku Puzzle")

- **Description:** This use case represents the central logic of the program, employing a recursive depth-first search with backtracking to brute-force all valid solutions to the loaded Sudoku puzzle.
- **Actor:** System (internal action).
- **Step-by-Step Outline:**
    1. The use case receives the 9x9 puzzle representation from the 'Solve Sudoku Puzzle' use case.
    2. System initiates a recursive function, starting from the first cell and searching for an empty square (_).
    3. For each empty square, the system iteratively tries digits 1 through 9.
    4. It checks the validity of each digit according to Sudoku rules (row, column, 3x3 box).

5. If a valid digit is found, the system places it and recursively calls the function for the next empty cell.
6. If no valid digit can be placed, or if all valid digits have been tried and the search must proceed, the system backtracks, removing the current digit and reverting the cell to empty, then exploring the next potential path.
7. If a complete 9x9 grid is filled according to the rules, a solution has been found. The use case continues to search for multi-part solutions.
8. Upon completing the search, it returns all found solutions (or indicates that no solution was found) to the primary use case.

- **Detailed Description:** This is where the core algorithmic work happens, implementing the DFS and backtracking mechanism as described. The recursive loop symbol within the oval effectively communicates the nature of the solution search. This process will systematically explore all possible completions, making it a complete, or exhaustive, search. The object-oriented approach would likely have methods within a 'SudokuSolver' class to handle this recursion. It is designed to find and store *all* solutions, addressing the potential for non-uniqueness.

Use Case 4: Display All Solutions (Extended by "Find Solutions")

- **Description:** This use case provides the specific functionality needed to store and format the results when the puzzle has more than one valid solution.
- **Actor:** System (internal action).
- **Step-by-Step Outline:**
    1. System finds a valid complete puzzle during the 'Find Solutions' use case.
    2. If this is the first solution found, it is added to a collection.
    3. If multiple solutions have been found, this use case is triggered to manage the complete set of solutions.
    4. Upon completion of the search, the collection of all unique solutions is returned to the system for subsequent display in the 'Print Output' use case.
- **Detailed Description:** This use case explicitly handles the management of non-unique solutions. While 'Find Solutions' explores the search space, 'Display All Solutions' ensures that every valid solution is captured and prepared for output. This prevents overwriting or losing intermediate solutions. When multiple solutions are present, this function might trigger the addition of numbering to the solution output to make it clear and organized, as suggested by "Display extra functionality when multi-part solutions exist." The 'extend' relationship to 'Find Solutions' is technically accurate, as finding a multi-part solution *is* an extended scenario that the system is equipped to handle.

Use Case 5: Print Output (Included by "Solve Sudoku Puzzle")

- **Description:** This use case manages the presentation of the original puzzle and its solutions (or lack thereof) to the user, directing this information to the console or other specified output.
- **Actor:** System (internal action, connecting to a 'Console/Output' secondary actor).
- **Step-by-Step Outline:**
    1. The use case receives the puzzle file name, the original puzzle, and the collection of solutions from the 'Solve Sudoku Puzzle' use case.
    2. System prints the puzzle file name (e.g., `puzzle1.txt`) to the console.
    3. System prints the original puzzle, displaying the filled digits and underscores.
    4. If solutions were found, the system iterates through them and prints each complete puzzle. If multiple solutions exist, they will all be printed sequentially.
    5. If no solution was found, the system prints the specific message "No solution found."
    6. This use case directs all this content to the external 'Console/Output' destination.
- **Detailed Description:** This use case ensures the output meets all specified requirements in a consistent format for each puzzle processed. By centralizing the output formatting, the program is made more maintainable. The diagram explicitly shows this use case connected to an external 'Console/Output' icon, indicating that this function is responsible for the actual display of the results to the user on their interface. This use case is critical for providing the user with meaningful feedback on the success of the solution attempt for each puzzle file.